Ingegneria del Software
Appunti e Approfondimenti

2024-2025

Contents

1 Definizione di Software **3**
- 1.1 Definizione . . . 3
- 1.2 (Alcune) Classificazioni dei Software . . . 3
  - 1.2.1 Software di sistema . . . 3
  - 1.2.2 Software applicativo . . . 3
  - 1.2.3 Software scientifico e tecnico . . . 3
  - 1.2.4 Software Embedded . . . 4
  - 1.2.5 Software di Produttività . . . 4
  - 1.2.6 Applicazioni Web . . . 4
  - 1.2.7 Software per l'Intelligenza Artificiale . . . 4
- 1.3 4P dello sviluppo software . . . 4
  - 1.3.1 People - Persone . . . 4
  - 1.3.2 Product - Prodotto . . . 5
  - 1.3.3 Project - Progetto . . . 5
  - 1.3.4 Process - Processo . . . 5
- 1.4 Ciclo di vita del software . . . 5

2 Processi di sviluppo **6**
- 2.1 Prospettive sul Processo . . . 6
  - 2.1.1 Activity perspective . . . 6
  - 2.1.2 Workflow perspective . . . 6
  - 2.1.3 Data-flow perspective . . . 6
  - 2.1.4 Role-action perspective . . . 6
- 2.2 Modelli di processo . . . 7
  - 2.2.1 Modello a cascata . . . 7
  - 2.2.2 Modello evolutivo . . . 7
  - 2.2.3 Modelli Model-Driven . . . 8
  - 2.2.4 Modelli iterativi . . . 8

3 Processo Unificato **9**
- 3.1 Struttura dell'UP . . . 9
  - 3.1.1 Avvio (o Inception) . . . 10
  - 3.1.2 Elaborazione (o Elaboration) . . . 10
  - 3.1.3 Costruzione (o Construction) . . . 11

3.1.4 Transizione (o Transition) 11
3.2 Casi d'Uso 12
3.2.1 Come individuare gli attori? 12
3.2.2 Descrizione dei Casi D'Uso 13
3.2.3 Test degli UC 14

4 Requisiti **15**
4.1 Definizione di Requisito 15
4.2 Tipi di Requisito 15
4.3 Ingegneria dei Requisiti 16
4.3.1 Elicitazione dei Requisiti 16
4.3.2 Tecniche di elicitazione 17
4.3.3 Tecniche Accessorie 17
4.3.4 Intervista 18
4.3.5 Workshop 18
4.3.6 KJ Method 19
4.4 Formato di definizione dei requisiti 20

5 UML **21**
5.1 Diagrammi Comportamentali 22
5.1.1 Use Case Diagram 22
5.1.2 Sequence Diagram 24
5.2 Diagramma di Attività 25

Chapter 1

Definizione di Software

1.1 Definizione

Un **software** è un programma che speficica le istruzioni che un calcolatore deve eseguire al fine di raggiungere uno scopo. Nel contesto dell'Ingegneria del Software, si includono nella definizione anche tutti i documenti che descrivono lo stesso nelle varie fasi della produzione del sistema.
L'**Ingegneria del Software** è la disciplina che agisce globalmente su tutte le componenti software (inclusi i documenti) al fine di produrre software definibile "di qualità".

1.2 (Alcune) Classificazioni dei Software

1.2.1 Software di sistema

I **Software di Sistema** sono programmi che gestiscono l'hardware di un computer e forniscono servizi di base per far funzionare altri software.
ES: Sistemi operativi, driver di dispositivo

1.2.2 Software applicativo

I **Software Applicativi** sono programmi usati dall'utente finale per svolgere compiti specifici.
ES: Microsoft Word, Google Chrome

1.2.3 Software scientifico e tecnico

I **Software Scientifici e Tecnici** sono progettati per applicazioni scientifiche, ingegneristiche o matematiche complesse.
ES: Matlab, AutoCAD

1.2.4 Software Embedded

I **Software Embedded** sono software incorporati in dispositivi hardare specifici, con funzionalità spesso limitate e progettati per uno scopo specifico.
ES: sistema operativo di una TV, firmware di una lavatrice

1.2.5 Software di Produttività

I **Software di Produttività** sono progettati per aiutare gli utenti a svolgere compiti legati al lavoro o alla gestione del tempo e delle risorse.
ES: Excel, Notion

1.2.6 Applicazioni Web

Software accessibili via browser, eseguiti da un server in remoto e che non richiedono installazione locale.
ES: Gmail, Facebook

1.2.7 Software per l'Intelligenza Artificiale

Software progettati per simulare capacità umane come apprendimento, ragionamento o comprensione del linguaggio.
ES: ChatGPT, Alexa

1.3 4P dello sviluppo software

1.3.1 People - Persone

Le persone coinvolte nel progetto o che ne traerebbero vantaggio. Definite anche "stakeholder" o "attori", possono essere a grandi linee divise in:

- **Business Management** - dirigenti, responsabili aziendali o chi ha una visione strategia del progetto. Stabiliscono gli obiettivi, decidono budget e tempistiche, approvano o fermano progetti in base ai risultati.
- **Project Manager** - hanno la responsabilità di organizzare e supervisionare il progetto giorno per giorno. Coordinano le attività del team, gestiscono tempi, costi, rischi e qualità, producono report e roadmap e mediano tra le esigenze tecniche e le richieste del cliente.
- **Development Team** - sviluppatori, designer, tester, analisti che lavorano direttamente alla realizzazione del software. Sviluppano il codice, testano le funzionalità, risolvono problemi tecnici e partecipano alla progettazione tecnica.
- **Customers** - clienti, persone o aziende che commissionano il software o che pagano per averlo sviluppato. Definiscono requisiti e aspettative, forniscono input funzionali e valutano il prodotto.

- **End Users** - utenti finali, persone che realmente usano il software nel loro quotidiano. Utilizzano il prodotto in casi reali e possono fornire feedback.

1.3.2 Product - Prodotto

Il Prodotto è l'insieme di funzionalità e servizi che il team consegna agli utenti o ai clienti, nonché l'obiettivo finale dello sviluppo.

1.3.3 Project - Progetto

Nel Progetto vengono specificate le attività da svolgere per realizzare il prodotto e i limiti a cui le stesse potrebbero incorrere. Fanno parte del progetto concetti come scope, budgeting, deadline, milestone e rischi.

1.3.4 Process - Processo

Il processo definisce quali sono e come organizzare le attività necessarie alla realizzazione di un software. Il Capitolo 2 si concentrerà sulla specifiche di differenti prospettive per l'analisi dei processi e su alcuni modelli specifici.

1.4 Ciclo di vita del software

Il **ciclo di vita** di un software è l'insieme di stati che lo stesso attraversa dal suo concepimento iniziale fino alla sua dimissione.
Le fasi principali del ciclo di vita sono:

1. **comunicazione**, identificazione iniziale delle funzionalità necessarie;
2. **pianificazione**, valutazione dei rischi e dei costi;
3. **modellazione**, progettazione delle funzionalità attraverso opportuni modelli;
4. **costruzione**, implementazione delle scelte e verifica delle stesse;
5. **deployment**, rilascio nell'ambiente;
6. **manuntenzione**, operazioni post-rilascio, possono essere di tre tipi:
   (a) **correttiva**, correzione di bug ed errori tecnici;
   (b) **adattiva**, adattamento del sistema a piattaforme o tecnologie differenti;
   (c) **perfettiva**, introduzione di nuove funzionalità;

Chapter 2

Processi di sviluppo

2.1 Prospettive sul Processo

Ogni processo di sviluppo può essere definito attraverso quattro prospettive differenti:

2.1.1 Activity perspective

Domanda: cosa deve essere fatto?
La **Activity Perspective** definisce le attività elementari che vanno a definire il processo. Ogni attività è definita in termini di input, output, condizioni di avvio e condizioni di completamento.

2.1.2 Workflow perspective

Domanda: in che ordine devono essere svolte le attività?
La **Workflow perspective** definisce la sequenza logica e temporale in cui vengono svolte le attività e le possibili alternative.

2.1.3 Data-flow perspective

Domanda: che artefatto produce ogni attività?
La **Data-flow perspective** (o **Artifact perspective**) definisce la sequenza di artefatti che vengono prodotti dal processo.

2.1.4 Role-action perspective

Domanda: chi fa cosa?
La **Role-Action perspective** individua team o persone incaricate di una determinata mansione.

2.2 Modelli di processo

I **modelli di processo** sono rappresentazioni schematiche o documentate dei flussi di lavoro necessari allo sviluppo di un sistema. Sono da immaginarsi come un'astrazione dei processi.

2.2.1 Modello a cascata

Il **modello a cascata** (o **waterfall**) è un modello di processo sequenziale e lineare, in cui ogni fase del processo deve essere terminata prma di passare alla successiva, senza possibilità di tornare indietro. Il suo nome deriva dal caratteristico Diagramma di Gantt.

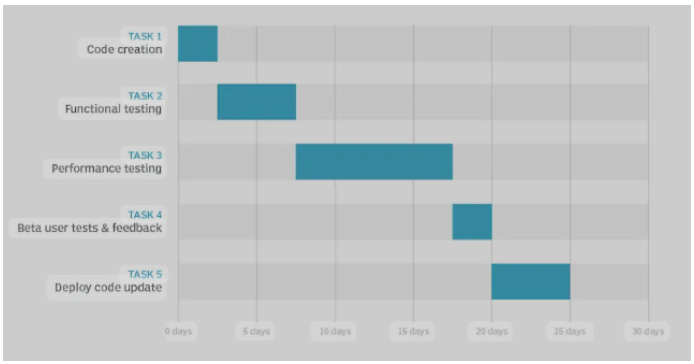


Figure 2.1: Esempio di diagramma di Gantt in un processo waterfall

Vantaggi Struttura semplice e di facile gestione, chiarezza nella documentazione e nelle fasi, adatto a progetti con requisiti ben definiti e stabili.

Svantaggi Poco flessibile ai cambiamenti, feedback degli stakeholder quasi totalmente ignorato, non adatto a progetti complessi o inesplorati.

2.2.2 Modello evolutivo

Il **modello evolutivo** è un modello di sviluppo in cui il prodotto viene costruito e migliorato progressivamente, seguendo il feedback degli utenti coinvolti. Si procede per versioni (**prototipi**) gradualmente sempre più completi. E' anche possibile realizzare prototipi col presupposto di cestinarli (**Throwaway prototyping**).

Vantaggi Alta flessibilità, feedback costanti, rischio di fallimento ridotto.

Svantaggi Possibile progettazione incompleta, difficile fare stima di costi e tempistiche.

2.2.3 Modelli Model-Driven

I **modelli model-drive** si basano sull'idea di sviluppare software partendo da modelli astratti, che poi vengono trasformati automaticamente o semi-automaticamente in codice eseguibile.

Vantaggi Scrittura del codice ridotta, riusco, coerenza e documentazione automatica favorite, manutenibilità più facile.

Svantaggi Richieste strumenti e competenze specifiche, complessità iniziale, tool non sempre perfetti.

2.2.4 Modelli iterativi

I **modelli iterativi** si pongono a metà tra i modelli a cascata e i modelli evolutivi. Prevedono lo sviluppo attraverso ripetizioni cicliche di mini-waterfall (dette **iterazioni**). In ogni iterazione si affrontano tutte le fasi del ciclo di vita, ma su un sottoinsieme limitato di requisiti. Alla fine di ogni iterazione il processo viene posto sotto valutazione degli stakeholder

Vantaggi Miglioramento continuo, facilità di correzione di errori, adattabilità ai cambiamenti nei requsiti.

Svantaggi Richiede maggior coinvolgimento del cliente, potenziale aumento dei costi.

Baseline Alla fine di ogni iterazione si genera un'insieme di artefatti rivisti e approvati chiamati **baseline**.

Rilascio incrementale La differenza tra la baseline dell'iterazione n e quella dell'iterazione $n-1$ viene chiamata **incremento**. Ad ogni incremento c'è un rilascio, interno od esterno, che permette di ottenere immediatamente il feedback da parte degli attori coinvolti ed adattare le future iterazioni di conseguenza. Questo processo prende il nome di **rilascio incrementale**.

Sviluppo a spirale In ogni iterazione vengono affrontati nuovi requisiti, si affinano le soluzioni e si riduce progressivamente l'incertezza, cercando di mitigare i rischi tecnici o di business all'inizio di ciascuna iterazione. Andando così a formare una sorta di modello a spirale, dove il rischio si riduce man mano.

Chapter 3

Processo Unificato

Il **processo unificato** (o **UP**, **Unified Process**) è un processo di sviluppo software iterativo con le seguenti caratteristiche:

Incentrato sull'architettura L'architettura del sistema viene definita dalle prime iterazioni.

Basato sul rilascio incrementale Concetto già spiegato nella sezione apposita.

Incentrato sul rischio Tutte le scelte, specialmente nelle fasi iniziali del processo, vengono prese prioritizzando la riduzione del rischio.

Guidato dai casi d'uso I casi d'uso sono usati per definire e specificare i requisiti.

Iterazioni time-boxed Le iterazioni seguono precise scadenze entro le quali devono essere concluse.

3.1 Struttura dell'UP

Lo Unified Process è strutturato a **fasi**. Una fase comprende più iterazioni. In ogni fase sono prioritirizzate alcune attività rispetto ad altre. Alla fine di ogni iterazione viene generata una baseline, questa viene valutata secondo alcune precise condizioni che, se verificate, dichiarano conclusa la fase. La baseline che verifica queste condizioni viene detta **milestone**. Le fasi del Processo Unificato sono **quattro**: avvio, elaborazione, costruzione e transizione.

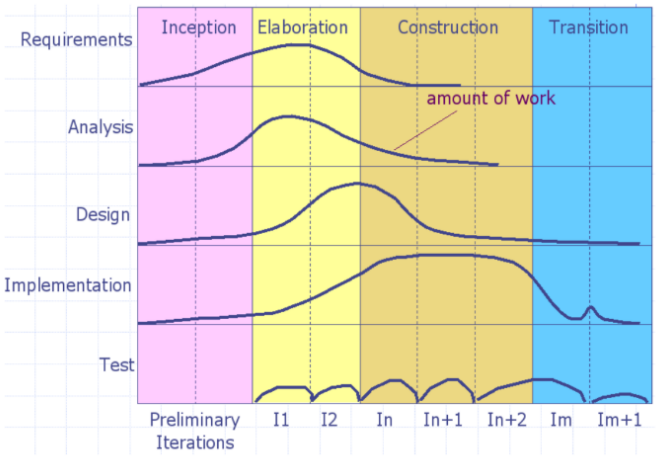


Figure 3.1: Distribuzione delle varie attività nelle fasi dell'UP

3.1.1 Avvio (o Inception)

Attività Principali Identificazione degli stakeholder, raccolta dei requisiti ad alto livello, definizione di uno Use Case Diagram iniziale, stima dei costi, analisi di fattibilità, valutazione dei rischi, definizione di un piano di progetto.

Obiettivi Definire ambito del sistema, identificare principali casi d'uso, valutare la fattibilità del progetto, stabilire un business case solido.

Milestone

- UC Diagram contenente 10-20% dei Casi D'Uso ben definiti
- Go-no go: decisione se il progetto è fattibile e vale la pena continuare
- Business case
- Vision document
- Lista ed analisi dei rischi

3.1.2 Elaborazione (o Elaboration)

Attività Principali Sviluppo dell'architettura software, completamento di use case significativi, definizione dell'architettura del sistema, inizio della documentazione tecnica, valutazione di rischi tecnici tramite prototipi. ù

Obiettivi Definire e validare architettura software, affinare i requisiti, ridurre i rischi più significativi, produrre un piano di progetto dettagliato per le fasi successive.

Milestone

- UC Diagram contenente 80-90% dei Casi D'Uso ben definiti
- Architettura stabile ed **eseguibile**
- Piano di progetto revisionato
- Stima dei rischi revisionata
- Accordo finale tra le parti firmato

3.1.3 Costruzione (o Construction)

Attività Principali Sviluppo iterativo del software completo, implementazione completa dei casi d'uso, scrittura, ottimizzazione, testing e documentazione del codice, gestione di eventuali cambiamenti nei requisiti, rivalutare i rischi.

Obiettivi Completare l'identificazione, la descrizione e l'implementazione dei Casi D'Uso, mantenere l'integrità dell'architettura del sistema.

Milestone

- Software stabile e di qualità sufficiente da essere rilasciato
- Verifica dei costi rispetto alla pianificazione iniziale
- Eventuali manuali o piani di distribuzione definiti nell'accordo tra le parti

3.1.4 Transizione (o Transition)

Attività Principali Testing lato utente, modifiche del design in caso siano emersi errori in fase di testing, supporto tecnico e documentazione, raccolta di feedback degli utenti, refactoring o ottimizzazione finale.

Obiettivi Rilascio prodotto agli utenti finali, correzione errori, addestramento e supporto agli utenti per l'utilizzo del software.

Milestone

- Prodotto software ultimato e testato
- Uso attivo del prodotto
- Definizione delle strategie di supporto (es. manuale utente)
- Piano di Manutenzione

3.2 Casi d'Uso

Un **Caso D'Uso** (o **CU** o **Use Case** o **UC**) è un insieme di scenari volti al descrivere un'interazione tipica tra un attore e il sistema per raggiungere un obiettivo specifico. I casi d'uso rappresentano le funzionalità osservabili dall'esterno e descrivono come il sistema risponde agli input degli attori (**passività del sistema rispetto all'attore**). Nel contesto del Processo Unificato, i Casi d'Uso sono il principale strumento per la definizione e specifica dei requisiti del sistema.

3.2.1 Come individuare gli attori?

Il primo passo per l'individuazione e definizione dei casi d'uso e l'individuazione degli attori che andranno ad attivarli. Un **attore** rappresenta un ruolo che un'entità può assumere. E' importante notare che un'entità può assumere più ruoli e quindi impersonare più attori. L'individuazione degli attori avviene attraverso una serie di domande relative alle funzioni che il team si pone:

- Chi/cosa usa il sistema?
- Chi partecipa alle varie fasi del ciclo di vita del sistema?
- Chi ottiene informazioni dal sistema/chi ne fornisce?
- Ci sono funzioni o azioni che vengono eseguite ad intervalli prestabiliti? ($\rightarrow$ Attore Speciale "Tempo")
- Ci sono eventi che modificano o attivano determinati comportamenti del sistema? Chi ne è responsabile?
- Per ogni funzionalità ideata, chi la usa? Chi la subisce? Chi ne beneficia?
- Esistono ruoli con privilegi diversi all'interno del sistema?
- Chi gestisce il sistema?

Un attore individuato viene considerato **valido** se ha una relazione con le funzionalità del sistema, cioè partecipa attivamente o passivamente ai flussi dei casi d'uso.
Una volta definiti, gli attori possono essere divisi in due categorie a seconda della loro relazione col sistema:

- **primari**, interagiscono direttamente col sistema e attivano casi d'uso
- **secondari**, subiscono l'effetto di casi d'uso o ne supportano la realizzazione

3.2.2 Descrizione dei Casi D'Uso

- **formato testuale (breve)**, riepilogo in un solo paragrafo in linguaggio naturale relativo al solo scenario principale
- **formato testuale (informale)**, più paragrafi indicanti i vari scenari
- **formato dettagliato**, tutti i passi e le variazioni descritti nel dettaglio, aggiunta di maggiori specifiche

Descrizione dettaglia dei Casi D'Uso

La descrizione degli UCs segue il seguente formato:

- **ID**, identificatore univoco per ogni caso d'uso
- **Nome**, nome significativo (secondo il formato MoSCoW)
- **Descrizione**, breve descrizione del Caso d'Uso
- **Sequenza principale (di successo)**, sequenza di azioni previste per il realizzarsi del caso d'uso
- **Sequenza alternativa**, eventuali diramazioni o divergenze della sequenza principale
- **Sequenza d'errore**, sequenze che s'attivano in caso di errore nella sequenza principale
- **Pre-condizioni**, condizioni necessariamente vere perché il caso d'uso possa essere attivato
- **Post-condizioni**, condizioni attive una volta terminata la sequenza del caso d'uso
- **Attori**, attori coinvolti nel caso d'uso

Descrizione scenari

La descrizione degli scenari avviene attraverso il linguaggio naturale seguendo questo formato:

<numero step>. <Attore><azione da compiere>

Eventuali ramificazioni sono indicate dalla parola "Se":

<numero step>. Se <condizione>
<numero step>.1 <azione in caso true>
<numero step>.2 <azione in caso false>

E' possibile esprimere cicli usando le parole chiave **for** e **while**, esprimendo la condizione in linguaggi naturale, o con dei rimandi ai passi precedenti.

<numero step>. Ritorna allo step <numero step - 1>

3.2.3 Test degli UC

Il livello di granularità dei Casi D'Uso è variabile, per trovare quello più adatto esistono una serie di tecniche di testing per valutare la correttezza del livello di astrazione:

- **test del capo**, capire se il dettaglio è rilevante per gli stakeholder di più alto livello
- **test del processo di business elementare**, verificare che il caso d'uso rappresenti effettivamente un processo completo e rilevante per il contesto di business
- **test della dimensione**, verificare la dimensione del caso d'uso, cioè se il numero di passi nella sequenza è superiore o inferiore ad una certa soglia (in genere tra i 5 e i 10)
  - se risulta **superiore**, bisogna dividere il caso d'uso in più casi d'uso
  - se risulta **inferiore**, potrebbe essere incorporato ad un UC più grande o direttamente omesso

Chapter 4

Requisiti

Le attività di individuazione e specifica dei requisiti prendono il nome di **Ingegneria dei Requisiti**. Si tratta di una fase fondamentale dello sviluppo software, in quanto si definisce cosa, ed in una certa misura come, il sistema debba fare.
L'ingegneria dei requisiti è anche fondamentale per valutare i rischi associati al progetto e valutare se continuare o meno.

4.1 Definizione di Requisito

Un requisito è una **condizione** o **capacità** da soddisfare per l'utente per risolvere un problema o raggiungere un obiettivo.
I requisiti possono essere definiti in modo:

- **informale**, attraverso il linguaggio naturale
- **semiformale**, attraverso un'opportuna notazione grafica (es. UML)
- **formale**, attraverso rigorosi modelli matematici

4.2 Tipi di Requisito

I requisiti si possono classificare in base a:

- **destinatari**
  - **utente**, si rivolgono direttamente all'utente, usano un linguaggio naturale e diagrammi, astrazione di alto livello
  - **di sistema**, si rivolgono principalmente ai progettisti, dettaglio di alto livello

I requisiti vengono inizialmente concepiti come requisiti utente, così da poter essere valutati da stakeholder meno competenti o tecnici, vengono poi successivamente derivati i corrispettivi requisiti di sistema. Entrambi i tipi di requisito riguardano comportamenti strettamente osservabili e non corrispondono a scelte strettamente di implementazione.

- **tipologia**
  - **funzionali**, indicano che servizi il sistema dovrebbe fornire, come dovrebbe reagire a determinati input e come dovrebbe comportarsi in generale. In generale rientrano in questa categoria tutti i requisiti che indicano funzionalità del sistema
  - **qualitativi (o non-funzionali o qualità)**, definiscono proprietà specifiche del sistema misurate secondo precise metriche
  - **vincoli**, requisiti che restringono, per ragioni organizzative o tecnologiche, il modo in cui il sistema deve essere sviluppato
  - **di dominio**, riguardano da requisiti derivati direttamente dallo specifico dominio o contesto applicativo

4.3 Ingegneria dei Requisiti

L'Ingegneria dei requisiti è un processo diviso in 4 parti:

1. **studio di fattibilità**, si determina se convenga o meno proseguire con il sviluppo
2. **elicitazione dei requisiti**, vengono individuati stakeholder opportuni a cui vengono richieste le possibili funzionalità da implementare e potenziali problemi annessi
3. **validazione**, si rimuovono possibili problemi nella specifica dei requisiti
4. **gestione**, si fanno emergere, permettono e gestiscono modifiche ai requisiti

4.3.1 Elicitazione dei Requisiti

La seconda fase precedentemente descritta richiedere un maggiore approfondimento. Nella ricerca dei requisiti si può incappare in problemi relativi al linguaggio naturale o a processi mentali quali rimozione, distorsione, generalizzazione. La scoperta dei requisiti avviene attraverso comunicazione diretta con gli attori, cercando di favorire la comunicazione con un gruppo eterogeneo piuttosto che omogeneo. A tale fine, si selezionando attori provenienti da gruppi divisi per:

- **punto di vista**
  - **primari**, interagiscono direttamente col sistema e attivano casi d'uso

  - **finali**, l'uso del sistema da parte di terzi realizza i loro obiettivi
  - **di supporto (o secondari)**, offrono un servizio al sistema
  - **fuoriscena**, sono interessati al comportamento del sistema ma non ci interagiscono direttamente.
- **ruolo**: persone, sistemi esterni, sensori
- **interfacce per l'interazione col sistema**

4.3.2 Tecniche di elicitazione

Per la scoperta dei requisiti si sfruttano specifiche tecniche. Le principali sono:

Focus group Una specifica parte del team approfondisce tematiche del sistema poco chiari.

Osservazione Si osservano i potenziali utilizzatori del sistema nello svolgimento delle regolari mansioni.

Questionari Viene distribuita agli stakeholder una lista di domande per comprendere concetti specifici.

Perspective-based Reading Identificazione e studio di documenti di rilievo per lo sviluppo.

Intervista Approfondimento.

Workshop Approfondimento.

4.3.3 Tecniche Accessorie

Sono inoltre spesso usate, in combinazione con le tecniche sopra elencate, alcune tecniche accessorie quali:

Brainstorming Emersione di idee innovative.

KJ Method Brainstorming in contesti con partecipanti eterogenei. Approfondimento.

Prototyping Concretizzazione pragmatica di alcuni aspetti del sistema.

Mind Mapping Costruzione di mappe concettuali.

Elicitation checklist Lista di tipici aspetti in un particolare contesto applicativo.

4.3.4 Intervista

L'**Intervista** è una tecnica in cui l'analista dei requisiti conversa con un gruppo di stakeholder per esplorare le sue necessità o aspettative e i vincoli a cui il sistema deve essere soggetto.
Ci sono tre tipi di interviste:

- **standardizzate**, con domande chiuse
- **esplorative**, con domande aperte
- **destrutturate**, con una conversazione attiva guidata dall'analista

L'intervista si compone di più fasi:

- **preparazione**, definizione delle modalità di intervista, delle domande da porre, delle possibili problematiche e degli obiettivi su cui chiedere chiarimenti, scelta degli intervistati ed individuazione del luogo e del tempo in cui svolgere l'intervista.
- **esecuzione**, l'effettiva esecuzione dell'intervista, si divide ulteriormente in:
  - **apertura**, introduzione di obiettivi e motivazioni
  - **conduzione**, l'analista pone le domande, prende appunti e chiede chiarimenti su risposte poco chiare
  - **chiusura**, riassunto dei punti salienti emersi durante la conduzione
- **rielaborazione**, analisi delle risposte raccolte per estrarre requisiti

La tecnica dell'intervista permette un'esplorazione profonda del punto di vista di uno stakeholder, comprese necessità nascoste o di cui c'è poca consapevolezza. E' però soggetta alla necessità di tempo per l'organizzazione, non garantisce l'individuazione di tutti i possibili requisiti ed è soggetto a bias, tra i quali:

- **bias di selezione**, il gruppo selezionato dall'intervistatore potrebbe non avere un'idea completa degli obiettivi o dei limiti del sistema
- **bias di conferma**, l'intervistatore potrebbe inconsciamente porre domande che riconfermino idee o aspettative già formate
- **bias di interpretazione**, in fase di rielaborazione potrebbero sorgere ambiguità dovute ad errori di interpretazione

4.3.5 Workshop

Il workshop è una tecnica di elicitazione basata sulla collaborazione tra un gruppo di stakeholder e l'analista. Si divide in tre fasi:

- **apertura**, si descrivono obiettivi, regole e possibili tecniche ausiliarie usate

- **conduzione**, avviene similarmente a quella dell'intervista, ma nel workshop la figura dell'analista viene accompagnata a quella del minute-taker, che trascrive tutto ciò che viene affermato durante la riunione in appunti chiamati "minutes"

- **chiusura**, stabilita dall'analista, ne segue la raccolta dei risultati

In seguito alla chiusura, il **minute-taker** riordina gli appunti presi e richiede feedback dei vari partecipanti al workshop, modificando i risultati a seconda dei loro responsi.
Alla luce di ciò, la principale caratteristica del workshop è la partecipazione attiva di più stakeholder e il confronto diretto tra essi e il minute-taker. Inoltre, permette facilmente di affiancare varie tecniche accessorie e di raccogliere in modo efficace più prospettive, identificando anche eventuali conflitti tra le varie visioni degli stakeholder. D'altro canto, richiede un lavoro maggiore, in quanto è spesso richiesto di filtrare, ordinare ed analizzare idee emerse, oltre che richiedere una disponibilità costante da parte degli stakeholder.

4.3.6 KJ Method

Il **KJ Method** è una tecnica accessoria usata per organizzare e raggruppare un gran numero di idee emerse. La tecnica consiste nella generazione di idee o input individuale (in genere trascritte su oggetti fisici come post-it o note) volte alla risoluzione di una problematica o al raggiungimento di un obiettivo. Una volta terminata la fase di riflessione individuale, le idee vengono esposte e lette dall'intero gruppo, si procede poi per raggruppamento delle idee simili, sintetizzandone con una breve etichetta il contenuto. Si votano i temi più rilevanti, così da darli priorità.
Il KJ Method permette l'emergere di requisiti lasciati implicitati, temi ricorrenti o ritenuti prioritari ed eventuali correlazioni tra esigenze diverse. Richiede, però, che siano disponibili input sufficienti da raggruppare e che sia presente una moderazione per evitare bias in fase di clustering.
Non rappresenta una tecnica di elicitazione completa, ma uno strumento di supporto con cui chiarire ed ordinare idee già emerse.

4.4 Formato di definizione dei requisiti

Esistono diversi formati per la definizione dei requisiti.
Uno dei più diffusi è il formato VOLERE, composto da questi campi:

NOME	**DESCRIZIONE**
Req. #ID	identificativo unico
Tipo	tipo di requisito
Evento	scenario del requisito
Descrizione	breve descrizione
Motivazione	perché il requisito è necessario
Origine	chi ha richiesto il requisito
Criterio di soddisfacimento	come verificare che il requisito è soddisfatto
Dipendenze e conflitti	rapporti con altri requisiti
Priorità	importanza rispetto agli altri requisiti
Storia	cronologia di creazione e modifiche

Chapter 5

UML

UML (Unified Modeling Language) è un linguaggio di modellazione standard utilizzato per analizzare, progettare e documentare sistemi software. UML permette di rappresentare graficamente la struttura e il comportamento del sistema, comunicare in modo chiaro tra tutte le figure partecipanti allo sviluppo e documentare le decisioni di analisi e design in modo coerente.
UML mette a disposizione 14 tipi di diagrammi per la modellizzazione del sistema e descrizione del suo comportamento.

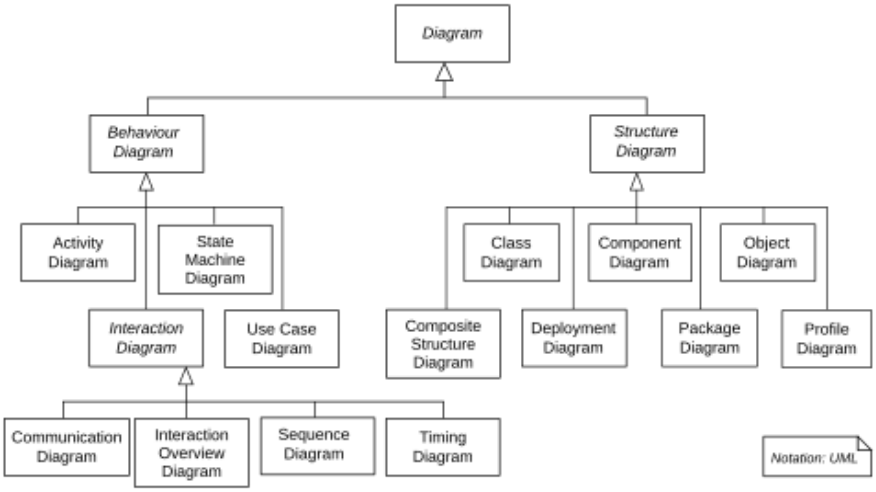


Figure 5.1: Organizzazione dei diagrammi in UML.

5.1 Diagrammi Comportamentali

5.1.1 Use Case Diagram

I meccanismi base di individuazione e definizione dei casi d'uso son già stati esplicati nella sezione dedicatali. In questa sezione pertanto ci si limiterà a presentare il diagramma dei casi d'uso, tenendo conto della sua notazione grafica e di eventuali meccanismi avanzati legati all'uso degli UC.
I diagrammi dei casi d'uso (o **Use Cases Diagram**) sono uno strumento dell'UML utilizzato per rappresentare tutti i casi d'uso presenti nel sistema e gli attori che li attivano.
L'obiettivo è mostrare quali sono le funzionalità del sistema, senza entrare nei dettagli implementativi del come vengono ottenute. Ogni UC descrive un'interazione significativa tra un attore e il sistema stesso, finalizzata a raggiungere un certo obiettivo.
Un diagramma dei casi d'uso è quindi uno dei primi diagrammi da sviluppare in sviluppo ed è composto da:

- **Attori** che rappresentano chi interagisce con il sistema attivando i casi d'uso. Sono rappresentati da uno stickman.
- **Casi d'uso** che rappresentano le funzionalità fornite dal sistema, rappresentati da un ovale.
- **Relazioni** tra attori e casi d'uso (attivazione) e tra casi d'uso stessi (inclusione, estensione, generalizzazione). Rappresentati attraverso delle linee.
- **Dominio** limiti del sistema o componente a cui sono associati determinati casi d'uso. Rappresentati da rettangoli.

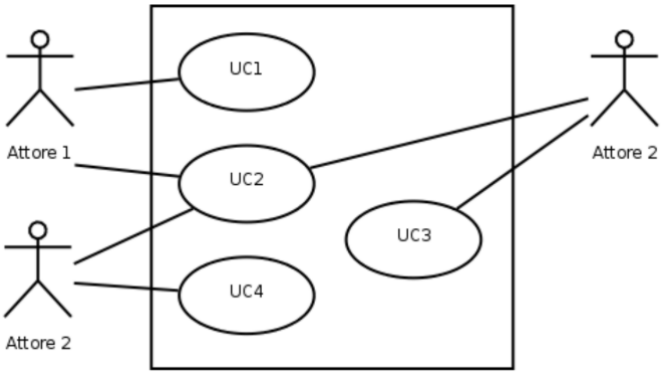


Figure 5.2: Rappresentazione base di un diagramma dei casi d'uso.

Meccanismi avanzati

UML mette a disposizione alcuni meccanismi avanzati per la rappresentazione degli UC.
Nello specifico:

- **Generalizzazione degli attori**, attori con comportamenti o UC attivabili in comune possono essere generalizzati da un unico attore.
- **Generalizzazione tra casi d'uso**, i casi d'uso possono ereditare, aggiungere o ridefinire comportamenti da altri UC.
- **Inclusione** <<**include**>>, riutilizzare comportamenti comuni tra casi d'uso.
- **Estensione** <<**extend**>>, permette di aggiungere funzionalità opzionali o condizionali.

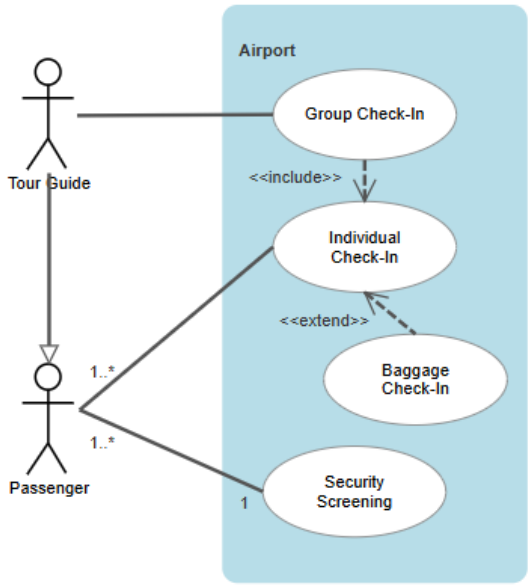


Figure 5.3: Esempio di Use Cases Diagram che fa uso dei meccanismi avanzati.

Ad esempio, nella Figure 5.3, l'attore Tour Guide è una composizione di attori Passenger. L'attore Tour Guide può inoltre attivare un solo caso d'uso: Group Check-in. Quest'ultimo caso d'uso include al suo interno il caso d'uso Individual Check-in, che a sua volta viene esteso da Baggage Check-in.

5.1.2 Sequence Diagram

I **diagrammi di sequenza** (o Sequence Diagram) descrivono attraverso notazione grafica come gli oggetti interagiscono tra di loro per realizzare un certo comportamento. Sono composti da:

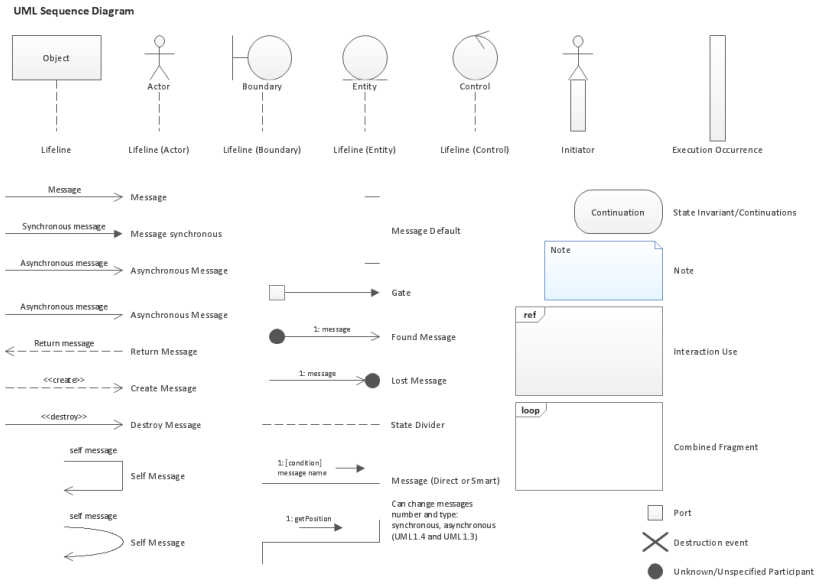


Figure 5.4: Notazione grafica dei vari elementi del Sequence Diagram.

- **Linee di vita (o lifelines)**, oggetti o classi coinvolte nell'interazione, sono rappresentati da un rettangolo contenente le informazioni base (nome, tipo) e da una linea verticale rappresentante la durata dell'esistenza nel sistema.
- **Messaggi**, sono l'interazione tra due lifeline. Possono essere:
  - Chiamate di operazione
  - Risposte
  - Chiamate di creazione/distruzione

Ulteriori approfondimenti sulle norme per realizzare i Sequence Diagram nell'apposita appendice.

5.2 Diagramma di Attività

I **Diagrammi di Attività** (o **Activity Diagram**) servono a rappresentare graficamente lo scenario principale di un caso d'uso e le sue varianti, visualizzare la sequenza di azioni e le decisioni prese dagli attori ed individuare punti di parallelismo.
La loro semantica è basata sulle Reti di Petri, dove il flusso di controllo è rappresentato da token che si spostano lungo gli archi.

Tipologie di nodi Gli elementi di un diagramma di attività sono:

- **Nodi azione**, operazioni elementari
- **Nodi di controllo**, gestiscono il flusso dei token
- **Nodi oggetto**, rappresentano dei buffer di dati presenti nel flusso
- **Flussi di controllo**, gestiscono il flusso logico attraverso i token
- **Flussi di oggetti**, gestiscono il flusso di dati tra le varie azioni

Nodi principali I principali nodi dell'Activity Diagram sono:

- **start node**, definisce dove inizia il flusso
- **action node**, indica una specifica azione, quando l'azione è terminata viene rilasciato un token
- **final flow node**, nodo di terminazione per quel specifico flusso
- **final activity node**, nodo di terminazione per l'intera attività, in caso di parallelismo anche gli altri flussi vengono interrotti
- **decision node**, permette la creazione di sequenze alternative
- **merge node**, definisce dove la sequenza alternativa si riunisce a quella principale
- **fork**, divide il flusso in più sotto-flussi che avanzano in parallelo
- **join**, termina il parallelismo

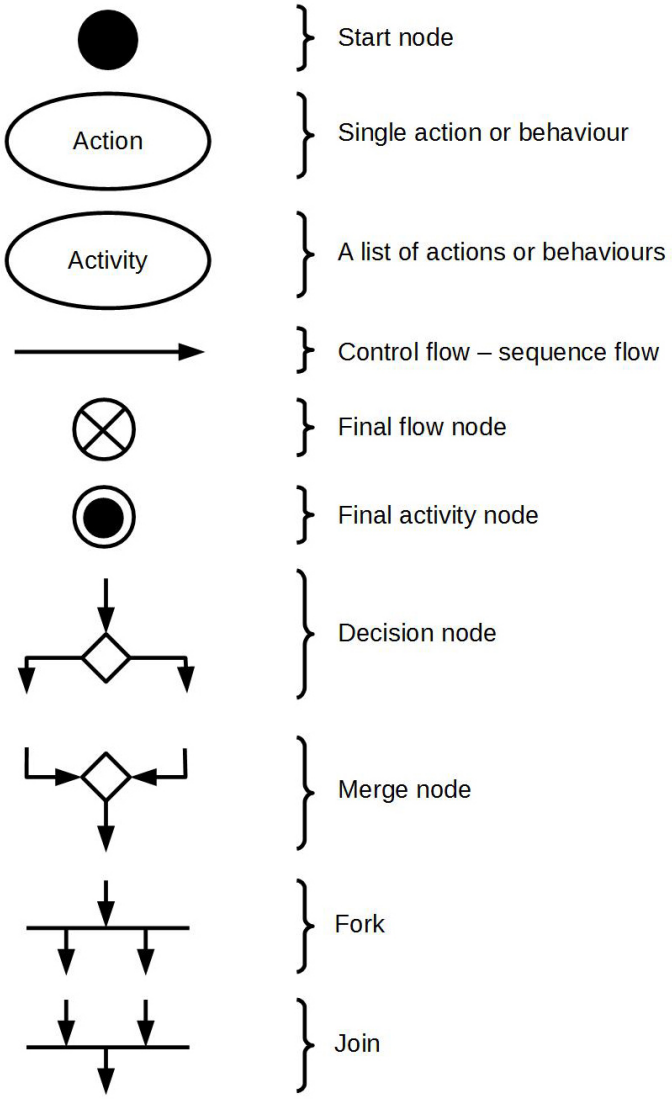


Figure 5.5: Notazione grafica dei vari elementi di un Activity Diagram.